PL/SQL Error Handling

- Exceptions (PL/SQL runtime errors) can arise from
  - Design faults
  - Coding mistakes
  - Hardware failures
  - Many other sources
- A developer
  - Cannot anticipate all possible exceptions
  - Can write exception handlers that lets the program continue to operate in their presence
- Any PL/SQL block can have an exception-handling part:

EXCEPTION

- The exception-handling part can have one or more exception handlers

PL/SQL Error Propagation

- When an exception is raised in the executable part of the block
  - The executable part stops
  - Control transfers to the exception-handling part
- If there is no enclosing block:
  - If the exception handler is in a subprogram, then control returns to the invoker, at the statement after the invocation.
  - If the exception handler is in an anonymous block, then control transfers to the host environment (for example, SQL*Plus)
- If there is an enclosing block:
  - Control transfers to the next statement of the enclosing block after an exception handler runs
  - The exception propagates if an exception is raised in a block that has no exception handler
  - The exception reproduces itself in successive enclosing blocks until a block has a handler for it or there is no enclosing block
  - If there is no handler for the exception, then PL/SQL returns an unhandled exception error to the invoker or host environment, which determines the outcome

PL/SQL Error Handling

- Exception categories:
  1. Internally defined
     - The runtime system raises internally defined exceptions implicitly (automatically).
     - Always has an error code
     - Does not have a name unless PL/SQL gives it one or you give it one
     - E.g.
       - ORA-00060 (deadlock detected while waiting for resource)
       - ORA-27102 (out of memory)

PL/SQL Error Handling

- The exception categories are:
  2. Predefined
     - A predefined exception is an internally defined exception that PL/SQL has given a name.
     - E.g.: ORA-06500 (PL/SQL: storage error) has the predefined name STORAGE_ERROR
  3. User-defined
     - Developers can declare their own exceptions in the declarative part of any PL/SQL
       - Anonymous block
       - Subprogram
       - Package
     - User-defined exceptions are raised explicitly
     - E.g.: Declare an exception named insufficient_funds to flag overdrawn bank accounts

PL/SQL Error Handling

Category	Definer	Has Error Code	Has Name	Raised Implicitly	Raised Explicitly
Internally defined	Runtime system	Always	Only if you assign one	Yes	Optionally
Predefined	Runtime system	Always	Always	Yes	Optionally
User-defined	User	Only if you assign one	Always	No	Always

- SQLCODE – function to get the error code
- SQLERRM – function to get the error message

PL/SQL Error Handling
Internally defined exception

- Internally defined exceptions (ORA-n errors)
  - Are described in Oracle Database Error Messages Reference
  - The runtime system raises them implicitly (automatically).
  - Does not have a name unless either PL/SQL gives it one or a developer gives it one
  - Can be handled only with **OTHERS** exception handlers.
- To give a name to an internally defined exception, do the following in the declarative part of the appropriate anonymous block, subprogram, or package:
  - Declare the name
  - Associate the name with the error code of the internally defined exception

PL/SQL Error Handling
Internally defined exception example

```
INSERT INTO employees(first_name, last_name)
VALUES('Ion', 'Ionescu');
```


PL/SQL Error Handling
Internally defined exception example

```
1  DECLARE
2    insert_null_exp exception;
3    PRAGMA exception_init(insert_null_exp, -1400);
4  BEGIN
5    INSERT INTO employees(first_name, last_name)
6    VALUES('Ion', 'Ionescu');
7  EXCEPTION
8    WHEN insert_null_exp THEN
9      dbms_output.put_line('Error msg  '||SQLERRM);
10     dbms_output.put_line('Error code '||SQLCODE);
11   WHEN others THEN
12     dbms_output.put_line('Others error');
13 END;
14 /
```

PL/SQL Error Handling
Pragma

- A pragma is an instruction to the compiler that is processed at compilation time
- A pragma begins with the reserved word PRAGMA followed by the name of the pragma
- Some pragmas have arguments
- A pragma may appear before a declaration or a statement
- Additional restrictions may apply for specific pragmas
- The extent of a pragma's effect depends on the pragma
- A pragma whose name or argument is not recognized by the compiler has no effect.

PL/SQL Error Handling
Predefined exceptions

- Predefined exceptions are internally defined exceptions that have predefined names
- PL/SQL declares the names of the predefined exceptions globally in the package STANDARD
- The runtime system raises predefined exceptions implicitly (automatically)
- Developers can write exception handlers specifically for predefined exceptions because they have names
- To explicitly raise an exception use RAISE

PL/SQL Error Handling
Predefined exceptions example

```
DECLARE
  n number := &n;
BEGIN
  IF n < 0 THEN
    RAISE INVALID_NUMBER;  -- raise explicitly
  END IF;
EXCEPTION
  WHEN INVALID_NUMBER THEN
    dbms_output.put_line('n < 0');
    dbms_output.put_line('Error msg  '||SQLERRM);
    dbms_output.put_line('Error code '||SQLCODE);
  WHEN others THEN
    dbms_output.put_line('Others error');
END;
/
```


PL/SQL Error Handling
User-defined exceptions

- Developers can declare their own exceptions in the declarative part of any PL/SQL anonymous block, Subprogram, or Package
- An exception name declaration has this syntax:
  exception_name exception;
- Developers can attach to a user defined exception
  - An exception code (-20999 și -20000) using
    PRAGMA exception_init(exception_name, exception_code);
  - An exception message using the procedure
    RAISE_APPLICATION_ERROR(exception_code, exception_message [, TRUE | FALSE]);
    - **TRUE** – PL/SQL puts **exception_code** on top of the error stack
    - **FALSE (default) –** PL/SQL replaces the error stack with **exception_code** (i.e., all previous errors are replaced with the current one)

PL/SQL Error Handling
User-defined exceptions Example

```
DECLARE
  n number := &n;
  my_exp1 exception;
  my_exp2 exception;
  PRAGMA exception_init(my_exp1, -20100);
  PRAGMA exception_init(my_exp2, -20200);
BEGIN
  IF n >= 200  THEN
    RAISE my_exp1;
  ELSIF n >= 100 THEN
    raise_application_error(-20200, 'Numar > 200');
  END IF;
EXCEPTION
  WHEN my_exp1 THEN
    dbms_output.put_line('n >100');
    dbms_output.put_line('my_exp1 error msg  '||SQLERRM);
    dbms_output.put_line('my_exp1 error code '||SQLCODE);
   WHEN my_exp2 THEN
    dbms_output.put_line('n > 200');
    dbms_output.put_line('my_exp2 error msg  '||SQLERRM);
    dbms_output.put_line('my_exp2 error code '||SQLCODE);
  WHEN others THEN
    dbms_output.put_line('Others error');
END;
/
```


PL/SQL Static SQL

- Static SQL is a PL/SQL feature that allows SQL syntax to be used directly in a block
- PL/SQL static SQL statements (some have a modified syntax):
  - SELECT
  - INSERT
  - UPDATE
  - DELETE
  - MERGE
  - COMMIT
  - ROLLBACK
  - SAVEPOINT
  - SET TRANSACTION
  - LOCK TABLE

PL/SQL Static SQL

- A PL/SQL static SQL statement can have a PL/SQL identifier (names given to a PL/SQL object) wherever its SQL counterpart can have a placeholder for a bind variable
- The PL/SQL identifier must identify either a variable or a formal parameter
- Bind variables
  - Are used to reuse the query parse tree and execution plan when new values are given
  - Can be used in dynamic PL/SQL code, but that's not good practice!
  - Are a placeholder for values within the query.
  - Unlike substitution variables, these are not prompted for when you come to compile the code.

PL/SQL Dynamic SQL

```
DECLARE
    v_sql varchar2(200);
    v_dname departments.department_name%type;
    id_dept departments.department_id%type := &id;
BEGIN
    v_sql := 'SELECT department_name FROM departments WHERE department_id = :1';
    EXECUTE IMMEDIATE v_sql INTO v_dname USING id_dept;
    dbms_output.put_line(v_dname);
END;
/
```

PL/SQL Static SQL

```
DECLARE
    v_sql varchar2(200);
    v_dname departments.department_name%type;
    id_dept departments.department_id%type := &id;
BEGIN
    SELECT department_name
    INTO v_dname
    FROM departments
    WHERE department_id = id_dept;

    dbms_output.put_line(v_dname);
END;
/

-- SQL Engine:
SELECT department_name FROM departments WHERE department_id = :1;
```


PL/SQL Static SQL

- The values of some variables are undefined after PL/SQL code runs a DML statement:
  - After a FETCH or SELECT statement raises an exception
    - The values of the define variables after that statement are undefined.
  - After a DML statement that affects zero rows
    - The values of the OUT bind variables are undefined unless the DML statement is a BULK or multiple-row operation

PL/SQL Static SQL

- Static SQL includes these SQL pseudocolumns (for different types of objects):
  - CURRVAL and NEXTVAL
  - LEVEL
    - Hierarchical Query Pseudocolumns (SELECT queries with START WITH ... CONNECT BY)
    - Returns 1 for a root row, 2 for a child of a root, and so on.
  - OBJECT_VALUE
    - Returns system-generated names for the columns of an object table, XMLType table, object view, or XMLType view
    - Useful for
      - Identifying the value of a substitutable row in an object table
      - Creating object views with the WITH OBJECT IDENTIFIER clause.
  - ROWID - returns the physical address of the row
  - ROWNUM - returns a number indicating the order in which Oracle selects the row from a table or set of joined rows for each row returned by a query

PL/SQL Static SQL

- To access the values of a sequence in SQL statements:
  - CURRVAL pseudocolumn
    - Returns the current value of the sequence
  - NEXTVAL pseudocolumn
    - Increments the sequence
    - Returns the new value
- Use dot notation to reference these pseudocolumns :

  sequence_name.CURRVAL
  sequence_name.NEXTVAL
- Each time sequence_name.NEXTVAL is reference
  - The sequence is incremented immediately and permanently
  - Commit or roll back on the transaction will not decrement the sequence

PL/SQL Static SQL

- Use sequence_name.CURRVAL and sequence_name.NEXTVAL in a PL/SQL expression wherever NUMBER expression can be used.
- Note:
  - Using sequence_name.CURRVAL or sequence_name.NEXTVAL to provide a **default value for an ADT method** parameter causes a **compilation error**
  - **PL/SQL evaluates every occurrence** of sequence_name.CURRVAL and sequence_name.NEXTVAL
  - **SQL evaluates a sequence expression for every row** in which it appears.

Cursors

- A cursor is a pointer to a private SQL area that stores information about processing a specific SELECT or DML statement
- A session cursor lives in session memory until the session ends
- When the session ends the cursor ceases to exist
- Cursor types:
  - Implicit
    - Constructed and managed by PL/SQL
  - Explicit
    - Construct and manage by the developer

Cursors

- Cursor attributes
  - Are used to get information about any session cursor
  - Can be referenced in procedural statements, but not in SQL statements
- Query the dynamic performance view V$OPEN_CURSOR to list the session cursors that each user session currently has opened and parsed
- The number of cursors that a session can have open simultaneously is determined by:
  - The amount of memory available to the session
  - The value of the initialization parameter OPEN_CURSORS

Cursors

- Generally, PL/SQL parses
  - An explicit cursor only the first time the session opens it
  - A SQL statement (creating an implicit cursor) only the first time the statement runs
- All parsed SQL statements are cached
- A SQL statement is reparsed only if it is aged out of the cache by a new SQL statement
- PL/SQL does not reparse the associated query if an explicit cursor is closed and immediately reopened

Implicit Cursors

- An implicit cursor is a session cursor that is constructed and managed by PL/SQL
- PL/SQL opens an implicit cursor every time SELECT or DML statement is executed
- A developer:
  - Cannot control an implicit cursor
  - Can get information from its attributes.

Implicit Cursors

- The syntax of an implicit cursor attribute value is SQL%attribute
- An implicit cursor is also called a SQL cursor
- SQL%attribute always refers to the most recently run SELECT or DML statement
- If no such statement has run, the value of SQL%attribute is NULL
- An implicit cursor closes after its associated statement runs
- Its attribute values remain available until another SELECT or DML statement runs

Implicit Cursors

- The most recently run SELECT or DML statement might be in a different scope:
  - To save an attribute value for later use, assign it to a local variable immediately
  - Otherwise, other operations, such as subprogram invocations, might change the value of the attribute before testing it
- The implicit cursor attributes are:
  - SQL%ISOPEN Attribute: Is the Cursor Open?
  - SQL%FOUND Attribute: Were Any Rows Affected?
  - SQL%NOTFOUND Attribute: Were No Rows Affected?
  - SQL%ROWCOUNT Attribute: How Many Rows Were Affected?
  - SQL%BULK_ROWCOUNT
  - SQL%BULK_EXCEPTIONS

Implicit Cursors

- SQL%ISOPEN always returns FALSE
  - An implicit cursor always closes after its associated statement runs
- SQL%FOUND returns:
  - NULL if no SELECT or DML statement has run
  - TRUE if a SELECT statement returned one or more rows or a DML statement affected one or more rows
  - FALSE otherwise
- SQL%NOTFOUND returns:
  - NULL if no SELECT or DML statement has run
  - FALSE if a SELECT statement returned one or more rows or a DML statement affected one or more rows
  - TRUE otherwise

Implicit Cursors

- The SQL%NOTFOUND attribute is not useful with the PL/SQL SELECT INTO statement:
  - If the SELECT INTO statement returns no rows, PL/SQL raises the predefined exception NO_DATA_FOUND immediately, before you can check SQL%NOTFOUND.
  - A SELECT INTO statement that invokes a SQL aggregate function always returns a value (possibly NULL), thus the SQL%NOTFOUND attribute is always FALSE, so checking it is unnecessary.

Implicit Cursors

- SQL%ROWCOUNT returns:
  - NULL if no SELECT or DML statement has run
  - Otherwise, the number of rows returned by a SELECT statement or affected by a DML statement (an INTEGER)
- If a SELECT INTO statement without a BULK COLLECT clause returns multiple rows
  - PL/SQL raises the predefined exception TOO_MANY_ROWS and SQL%ROWCOUNT returns 1
  - This is not the actual number of rows that satisfy the query
- The value of SQL%ROWCOUNT attribute is unrelated to the state of a transaction
- The value of SQL%ROWCOUNT is not restored
  - To the value it had before the savepoint when a transaction rolls back to a savepoint
  - To the original value in the parent transaction when an autonomous transaction ends

Implicit Cursors

- To allow a FORALL statement to continue even if some of its DML statements fail, include the SAVE EXCEPTIONS clause
- Use SQL%BULK_EXCEPTIONS in the FORALL clause to find the element that caused an error
- Use SQL%BULK_ROWCOUNT to get the total number of rows affected by the FORALL statement

Implicit Cursors

```
set serveroutput on
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );
  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
  v_aa; -- create a null associative array
  err_idx PLS_INTEGER;
  err_count PLS_INTEGER;
BEGIN

  v := aa(1 => rec('RO', 'Romania', 1), 2 => rec('MD', 'Moldova', 1),
          5 => rec('BG', 'Bulgaria', 1));

  FORALL j IN v.FIRST .. v.LAST SAVE EXCEPTIONS
  INSERT INTO COUNTRIES VALUES v(j);

EXCEPTION
  WHEN others THEN
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
    err_count := SQL%BULK_EXCEPTIONS.COUNT;
    FOR i IN 1..err_count LOOP
      DBMS_OUTPUT.PUT_LINE ('SQL Code '||SQL%BULK_EXCEPTIONS(i).ERROR_CODE);
      err_idx := SQL%BULK_EXCEPTIONS(i).ERROR_INDEX;
      DBMS_OUTPUT.PUT_LINE('Bad statement #: ' || err_idx);
    END LOOP;

    FOR i IN v.FIRST..v.LAST LOOP
      DBMS_OUTPUT.PUT_LINE('INSERT '|| i ||' EXECUTETD? '||SQL%BULK_ROWCOUNT(i));
    END LOOP;

    ROLLBACK;
END;
/
```

Explicit Cursors

- An explicit cursor is a session cursor that the developer constructs and manages
- The developer
  - Declares and defines an explicit cursor
  - Gives the cursor name associating a query with it (typically, the query returns multiple rows).
  - Can process the query result set in either of these ways:
    - Using **OPEN-FETCH-CLOSE**
      - Open the explicit cursor (with the OPEN statement)
      - Fetch rows from the result set (with the FETCH statement)
      - Close the explicit cursor (with the CLOSE statement)
    - Use the explicit cursor in a cursor **FOR LOOP** statement

Explicit Cursors

- The developer cannot
  - Assign a value to an explicit cursor
  - Use a cursor in an expression
  - Use a cursor as a formal subprogram parameter or a host variable
- A developer can reference an explicit cursor or cursor variable by its name unlike an implicit cursor
- An explicit cursor or cursor variable is called a named cursor

Declaring and Defining Explicit Cursors

- The developer can either
  - Declare an explicit cursor first and then define it later in the same block, subprogram, or package,
  - Declare and define it at the same time.
- An explicit cursor declaration, which only declares a cursor, has this syntax:

CURSOR cursor_name [parameter_list] RETURN return_type;

- An explicit cursor definition has this syntax:

CURSOR cursor_name [parameter_list] [RETURN return_type] IS select_statement;

Opening an Explicit Cursors

- After declaring and defining an explicit cursor, the developer can open it using the OPEN statement
- The OPEN statement does the following:

1. Allocates database resources to process the query
2. Processes the query:
   - Identifies the result set
     - If the query references variables or cursor parameters, their values affect the result set
   - If the query has a FOR UPDATE clause, locks the rows of the result set
3. Positions the cursor before the first row of the result set

Fetching Data with Explicit Cursors

- After opening an explicit cursor, the developer can fetch the rows of the query result set with the FETCH statement
- The basic syntax of a FETCH statement that returns one row is:

FETCH cursor_name INTO into_clause;

- The **into_clause** is either a list of variables or a single record variable
- The variable list or record must have a corresponding type-compatible variable or field for each column that the query returns
- The %TYPE and %ROWTYPE attributes are useful for declaring variables and records for use in FETCH statements

Fetching Data with Explicit Cursors

- The FETCH statement
  - Retrieves the current row of the result set
  - Stores the column values of that row into the variables or record
  - Advances the cursor to the next row
- The developer can
  - Use the FETCH statement inside a LOOP statement
  - Exit the LOOP when the FETCH statement runs out of rows (use %NOTFOUND)
- PL/SQL does not raise an exception when a FETCH statement returns no rows

Closing an Explicit Cursors

- The developer can close an open explicit cursor with the CLOSE statement
- The CLOSE statement allows the allocated resources for the cursor to be reused.
- After closing a cursor, t**he developer cannot**
  - Fetch records from the cursor's result set
  - Reference the cursor's attributes.
  - PL/SQL raises the predefined exception INVALID_CURSOR otherwise
- The developer can reopen a closed cursor
- The explicit cursor must be closed before trying to reopen it.
  - PL/SQL raises the predefined exception CURSOR_ALREADY_OPEN otherwise

Explicit Cursors %ISOPEN and %ROWCOUNT Attributes

- %ISOPEN returns
  - TRUE if its explicit cursor is open
  - FALSE otherwise.
- %ISOPEN is useful for checking that an explicit cursor is not already open before you try to open it
- If a developer tries to open an explicit cursor that is already open PL/SQL raises the predefined exception CURSOR_ALREADY_OPEN
- %ROWCOUNT returns:
  - Zero after the explicit cursor is opened but before the first fetch
  - Otherwise, the number of rows fetched (an INTEGER)

Explicit Cursors
%FOUND and %NOTFOUND Attributes

- %FOUND returns:
  - NULL after the explicit cursor is opened but before the first fetch
  - TRUE if the most recent fetch from the explicit cursor returned a row
  - FALSE otherwise
- %FOUND is useful for determining whether there is a fetched row to process.
- %NOTFOUND returns:
  - NULL after the explicit cursor is opened but before the first fetch
  - FALSE if the most recent fetch from the explicit cursor returned a row
  - TRUE otherwise
- %NOTFOUND is useful for exiting a loop when FETCH fails to return a row

Explicit Cursors Example

```
set serveroutput on

DECLARE
  CURSOR c1 IS
    SELECT region_name, country_name cn, country_id
    FROM regions NATURAL JOIN countries ORDER BY 1, 2;
  CURSOR c2 RETURN locations%ROWTYPE IS
    SELECT *
    FROM locations
    ORDER BY city;
  r1 c1%ROWTYPE;
  r2 locations%ROWTYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    OPEN c2;
    LOOP
      FETCH c2 INTO r2;
      EXIT WHEN c2%NOTFOUND;
      IF r2.country_id = r1.country_id THEN
        DBMS_OUTPUT.PUT_LINE(r1.region_name
                             || '::' || r1.cn ||'::'||r2.city);
      END IF;
    END LOOP;
    IF c2%ISOPEN THEN
      CLOSE c2;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```


Variables in
Explicit Cursors Queries

- An explicit cursor query can reference any variable in its scope
- When opening an explicit cursor
  - PL/SQL evaluates any variables in the query
  - Uses those values when identifying the result set
- Changing the values of the variables later does not change the result set

Explicit Cursors Example

```
set serveroutput on
DECLARE
  idr regions.region_id%TYPE := &id;
  CURSOR c1 IS
    SELECT region_name rn, country_name cn, city
    FROM regions NATURAL JOIN countries NATURAL JOIN LOCATIONS
    WHERE region_id = idr
    ORDER BY 1, 2;
  r1 c1%ROWTYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```


Explicit Cursors
Parameters

- Developers can
  - Create an explicit cursor that has formal parameters
  - Pass different actual parameters to the cursor each time you open it
- A developer can use a formal cursor parameter anywhere a constant can be used in the cursor query
- A developer cannot reference formal cursor parameters outside the cursor query
- Use different names for formal and actual cursor parameters to avoid confusion
- A developer can specify a default value for formal parameters of a cursor

Explicit Cursors
Parameters Example

```
set serveroutput on
DECLARE
  CURSOR c1(idr regions.region_id%TYPE DEFAULT 1, idc varchar) IS
    SELECT region_name rn, country_name cn, city
    FROM regions NATURAL JOIN countries NATURAL JOIN LOCATIONS
    WHERE region_id = idr AND country_id = idc
    ORDER BY 1, 2;
  r1 c1%ROWTYPE;
BEGIN
  OPEN c1(idc => 'UK');
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;

  OPEN c1(idr=> 2, idc => 'US');
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```


Cursors FOR LOOP Statement

- The cursor FOR LOOP statement lets developers run a SELECT statement and then immediately loop through the rows of the result set
- This statement can use either an implicit or explicit cursor (but not a cursor variable)
- If only a SELECT statement is specified inside the FOR LOOP statement
  - This form of the cursor FOR LOOP statement uses an implicit cursor
  - Is called an implicit cursor FOR LOOP statement
  - The implicit cursor is internal to the statement
  - **The statement cannot be referenced with the name SQL**
- If an explicit cursor is specified inside the FOR LOOP statement
  - Is called an explicit cursor FOR LOOP statement

Cursors FOR LOOP Statement

- **The cursor FOR LOOP statement implicitly declares its loop index**
- The loop index is %ROWTYPE record variable of the type that the cursor returns
- This **index record is local to the loop and exists only during loop execution.**
- Statements inside the loop can reference the record and its fields
- They can reference virtual columns only by aliases

Cursors FOR LOOP Statement

- After declaring the loop index record variable, the FOR LOOP statement
  - Opens the specified cursor
  - Fetches a row from the result set and stores it in the record with each iteration of the loop
  - Closes the cursor when there are no more rows to fetch
- The cursor also closes if a statement inside the loop
  - Transfers control outside the loop (i.e., using GOTO)
  - PL/SQL raises an exception

Cursors FOR LOOP Statement Example

```
1 set serveroutput on
2 DECLARE
3   CURSOR c1 IS
4     SELECT region_name rn, country_name cn, country_id cid
5     FROM regions NATURAL JOIN countries ORDER BY 1, 2;
6 BEGIN
7   FOR r1 in c1
8   LOOP
9     FOR r2 IN (SELECT city c FROM locations
10              WHERE country_id=r1.cid ORDER BY city)
11    LOOP
12        DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r2.c);
13    END LOOP;
14  END LOOP;
15 END;
16 /
```


Cursor Variables

- A cursor variable is like an explicit cursor, except that:
  - It is **not limited to one query**
  - It can be used to open a cursor variable for a query, process the result set, and then use the cursor variable for another query.
  - **A value can be assigned to it**
  - It can be used in an **expression**
  - It can be **a subprogram parameter**
  - It can be used to **pass query result sets between subprograms**
  - It can be a **host variable**
  - It can be used to **pass query result sets between PL/SQL stored subprograms and their clients**
  - It **cannot accept parameters**
  - Whole queries can be passed to it (the queries can include variables)

Cursor Variables

- A cursor variable offers flexibility because it is a pointer
  - Its value is the address of an item, not the item itself
- Before a cursor variable can be referenced it must point to a SQL work area
  - By opening the SQL work area
  - By assigning it the value of an opened
    - PL/SQL cursor variable
    - Host cursor variable
- Cursor variables and explicit cursors are not interchangeable, one cannot be used where the other is expected
  - E.g., a cursor variable cannot be used in a cursor FOR LOOP statement.

Cursor Variable

- To create a cursor variable
  - Declare a variable of the predefined type SYS_REFCURSOR
  - Define a REF CURSOR type and then declare a variable of that type.
- Informally, a cursor variable is sometimes called a REF CURSOR
- The basic syntax of a REF CURSOR type definition is:
  TYPE type_name IS REF CURSOR [RETURN return_type]
- The REF CURSOR type and cursor variables of that type are strong if **return_type** is specified, otherwise, they are weak.
- SYS_REFCURSOR and cursor variables of that type are weak

Cursor Variable

- Only queries that return the specified type can be associated with a strong cursor variable
- Any query can be associated with a weak cursor variable
- Weak cursor variables are more error-prone than strong ones, but they are also more flexible.
- Weak REF CURSOR types are interchangeable with each other and with the predefined type SYS_REFCURSOR.
- The value of a weak cursor variable can be assigned to any other weak cursor variable
- The value of a strong cursor variable can be assigned to another strong cursor variable only if both cursor variables have the same type (not merely the same return type).

Assigning Values to a Cursor Variable

- PL/SQL cursor variable can be initialized with
  - Another PL/SQL cursor variable
  - A host cursor variable
- The syntax is:
  target_cursor_variable := source_cursor_variable;
- If source_cursor_variable is open, then after the assignment, target_cursor_variable is also open
- The two cursor variables point to the same SQL work area
- If source_cursor_variable is not open, opening target_cursor_variable after the assignment does not open source_cursor_variable

Variables in a Cursor Variable

- The query associated with a cursor variable can reference any variable in its scope
- When a cursor variable is opened with the OPEN FOR statement, PL/SQL
  - Evaluates any variables in the query
  - Uses those values when identifying the result set
- Changing the values of the variables later does not change the result set

Opening a Cursor Variable

- A cursor variable can be opened with the OPEN FOR statement

1. Associates the cursor variable with a query (typically, the query returns multiple rows)
   - The query can include placeholders for bind variables
   - The bind variable values can be specified in the USING clause of the OPEN FOR statement
2. Allocates database resources to process the query
3. Processes the query:
   - Identifies the result set
   - If the query references variables, their values affect the result set
   - If the query has a FOR UPDATE clause, locks the rows of the result set
4. Positions the cursor before the first row of the result set

Fetching Data with a Cursor Variable

- After opening a cursor variable the rows of the query result set can be fetched with the FETCH statement
- The return type of the cursor variable must be compatible with the INTO clause of the FETCH statement
- PL/SQL catches incompatibility
  - At compilation time if the cursor variable is strong
  - At run time if the cursor variable is weak
- The predefined exception ROWTYPE_MISMATCH is raised before the first fetch in case of incompatibility for a weak cursor variable

Closing a Cursor Variable

- A cursor variable needs to be closed before reopening it (that is, using it in another OPEN FOR statement)
- The query previously associated with a cursor variable is lost after it is reopened
- A cursor variable is closed with the CLOSE statement
- Using the CLOSE statement allows the resources associated with the cursor variable to be reused
- After closing a cursor variable
  - Its records cannot be fetched
  - Its result set cannot be accessed
  - Its attributes cannot be referenced
- Trying any of these raises the predefined exception INVALID_CURSOR

Cursor Variable Example

```
DECLARE
  TYPE tbl IS TABLE OF VARCHAR2(50);
  TYPE rc IS REF CURSOR RETURN departments%ROWTYPE;
  c1 rc;
  c2 SYS_REFCURSOR;
  r1 departments%ROWTYPE;
  names tbl;
BEGIN
  OPEN c1 FOR SELECT * FROM departments;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    OPEN c2 FOR SELECT first_name || ' ' || last_name ename
    FROM employees WHERE department_id=r1.department_id;
    FETCH c2 BULK COLLECT INTO names;
    IF names.COUNT > 0 THEN
      FOR i IN names.FIRST .. names.LAST LOOP
        DBMS_OUTPUT.PUT_LINE(r1.department_name || '::' || names(i));
      END LOOP;
    END IF;
    IF c2%ISOPEN THEN
      CLOSE c2;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```


Cursor Expressions

- A CURSOR expression returns a nested cursor
- It has this syntax:

  CURSOR (subquery)

- CURSOR expression can be
  - Used in a SELECT statement that is not a subquery
  - Passed to a function that accepts a cursor variable parameter
- A cursor expression cannot be used with an implicit cursor

Cursor Expressions Example

```
DECLARE
  CURSOR c1 IS
    SELECT department_name,
      CURSOR ( SELECT e.last_name || ' ' || e.first_name name
               FROM employees e
               WHERE e.department_id = d.department_id
               ORDER BY e.last_name
             ) employees
    FROM departments d
    ORDER BY department_name;
  c2 SYS_REFCURSOR;
  dname departments.department_name%TYPE;
  ename varchar2(45);
BEGIN
  OPEN c1;
  LOOP  -- Process each row of query result set
    FETCH c1 INTO dname, c2;
    EXIT WHEN c1%NOTFOUND;
    LOOP  -- Process each row of subquery result set
      FETCH c2 INTO ename;
      EXIT WHEN c2%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(dname||'::'||ename);
    END LOOP;
  END LOOP;
  CLOSE c1;
END;
/
```


FOR UPDATE

- The SELECT statement with the FOR UPDATE clause (SELECT FOR UPDATE statement) selects the rows of the result set and locks them
- SELECT FOR UPDATE
  - Locks rows for update
  - It ensures that no other user can change those values before an update
- By default, the SELECT FOR UPDATE statement waits until the requested row lock is acquired
- To change this behavior, use the NOWAIT, WAIT, or SKIP LOCKED clause of the SELECT FOR UPDATE statement.

FOR UPDATE

```
DECLARE
  CURSOR c1 IS
    SELECT employee_id, first_name, last_name, salary, department_id
    FROM employees
    WHERE employee_id NOT IN
      (SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL)
    FOR UPDATE OF salary
    ORDER BY department_id;
  asd number(10,2);
  r1 c1%ROWTYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    SELECT avg(salary) INTO asd FROM employees
    WHERE department_id = r1.department_id;
    IF r1.salary > asd THEN
      UPDATE employees SET salary = salary * 1.10
      WHERE
        CURRENT OF c1;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
  ROLLBACK;
END;
/

select salary from employees;
```

FOR UPDATE

- When SELECT FOR UPDATE is associated with an explicit cursor, the cursor is called a FOR UPDATE cursor
- Only a FOR UPDATE cursor can appear in the CURRENT OF clause of an UPDATE or DELETE statement.
- The CURRENT OF clause
  - Is a PL/SQL extension to the WHERE clause of the SQL statements UPDATE and DELETE
  - Restricts the statement to the current row of the cursor
- When SELECT FOR UPDATE queries multiple tables, it locks only rows whose columns appear in the FOR UPDATE clause.

Subprograms

- A PL/SQL subprogram is a named PL/SQL block that can be invoked repeatedly
- If the subprogram has parameters, their values can differ for each invocation
- A subprogram is either a
  - **Procedure** used to perform an action
  - **Function** used to compute and return a value
- Subprograms support the development and maintenance of reliable and reusable code

Subprograms Features

- Modularity
  - Subprograms let you break a program into manageable, well-defined modules
- Easier Application Design
  - Implementation details of the subprograms can be deferred until the main program is tested
  - The programs can be refined one step at a time
  - Easier to test modules independently
- Maintainability
  - The implementation details of a subprogram can be changed without changing its invokers
- Packageability
  - Subprograms can be grouped into packages

Subprograms Features

- Reusability
  - Any number of applications, in many different environments, can use the same package subprogram or standalone subprogram
- Better Performance
  - Each subprogram is compiled and stored in executable form, which can be invoked repeatedly
  - Because stored subprograms run in the database server, a single invocation over the network can start a large job
  - This division of work
    - Reduces network traffic
    - Improves response times
  - Stored subprograms are cached and shared among users
    - Lowers memory requirements
    - Lowers invocation overhead

Subprograms Types

- Subprogram can be
  - Nested subprograms:
    - Created inside the WITH clause
    - Created inside a PL/SQL block (which can be another subprogram)
  - Package subprograms: created inside a package
  - Standalone subprograms: are created at schema level

Nested Subprograms

- A subprogram created inside a PL/SQL block or WITH clause is a nested subprogram
- In the WITH clause
  - Declare **functions** that will be used in the SQL statement
  - Declare **procedures** that can only be used in subprograms
- Nested subprograms can be
  - Declared and defined at the same time
  - First declared and then defined in the same block
- A nested subprogram is stored in the database only if it is nested in
  - A standalone subprogram
  - Package subprogram

Nested Subprograms

```
WITH
  FUNCTION calc_venit(eid employees.employee_id%type)
  RETURN FLOAT
  IS
    sal employees.salary%type;
    com employees.commission_pct%type;
  BEGIN
    SELECT salary, commission_pct
    INTO sal, com
    FROM employees
    WHERE employee_id = eid;
    RETURN sal * (1 + nvl(com, 0));
  END;
SELECT
  first_name||' '||last_name nume,
  calc_venit(employee_id) venit
FROM employees;
/
```


Nested Subprograms

```
WITH
  PROCEDURE procent_comm(eid employees.employee_id%type, procent NUMBER)
  IS
    name varchar2(50);
  BEGIN
    SELECT first_name||' '||last_name INTO name
    FROM employees WHERE employee_id=eid;
    DBMS_OUTPUT.put_line(name || ' procent=' || procent);
  END;

  FUNCTION calc_venit(eid employees.employee_id%type)
  RETURN FLOAT
  IS
    sal employees.salary%type;
    com employees.commission_pct%type;
  BEGIN
    SELECT salary, commission_pct INTO sal, com
    FROM employees WHERE employee_id = eid;
    procent_comm(eid, 1 + nvl(com, 0));
    RETURN sal * (1 + nvl(com, 0))
  END;
SELECT
  first_name||' '||last_name nume,
  calc_venit(employee_id) venit
FROM employees;
/
```

Package Subprograms

- A subprogram created inside a package is a **package subprogram**
- Developers can
  - Declare it in the package specification
  - Define it in the package body
- It is stored in the database until the package is dropped

Standalone Subprograms

- **A subprogram created at schema level is a standalone subprogram**
- A developer can create it with the statements:

CREATE [OR REPLACE] FUNCTION function_name
CREATE [OR REPLACE] PROCEDURE procedure_name

- It is stored in the database until it is dropped with the statements:

DROP FUNCTION function_name;
DROP PROCEDURE procedure_name;

Stored Subprograms

- A stored subprogram is either
  - A package subprogram
  - A standalone subprogram
- A stored subprogram is affected by the AUTHID and ACCESSIBLE BY clauses
- These clauses can appear in the CREATE FUNCTION, CREATE PROCEDURE, and CREATE PACKAGE statements
- The AUTHID clause affects the name resolution and privilege checking of SQL statements that the subprogram issues at run time
- The ACCESSIBLE BY clause specifies a white list of PL/SQL units that can access the subprogram

Subprogram Invocations

- A subprogram invocation has this form:

subprogram_name [([parameter [, parameter]...])]

- If the subprogram has no parameters, or specifies a default value for every parameter
  - Omit the parameter list
  - Specify an empty parameter list
- A procedure invocation is a PL/SQL statement:

procedure_name(val_parameter_1, val_parameter_2, ...);

- A function invocation is an expression:

value := function_name(val_parameter_1, val_parameter_2, ...);

IF function_name(val_parameter_1, val_parameter_2, ...) THEN ...

Subprogram Properties

- Each subprogram property (e.g., AUTHID, ACCESSIBLE BY, etc.) can appear only once in the subprogram declaration
- The properties can appear in any order
- Properties appear before the IS or AS keyword in the subprogram heading
- The properties cannot appear in nested subprograms
- Only the ACCESSIBLE BY property can appear in package subprograms

Subprogram Parts

- A subprogram begins with a subprogram heading, which specifies
  - The subprogram name
  - The subprogram parameter list (optional)
- Like an anonymous block, a subprogram has these parts:
  - Declarative part (optional)
    - This part declares and defines local types, cursors, constants, variables, exceptions, and nested subprograms
    - These items cease to exist when the subprogram completes execution
    - This part can also specify pragmas
    - The declarative part of a subprogram does not begin with the keyword DECLARE, as the declarative part of an anonymous block does
  - Executable part (required)
    - This part contains one or more statements that assign values, control execution, and manipulate data
  - Exception-handling part (optional)
    - This part contains code that handles runtime errors

Additional Subprogram Parts For Functions

- A function has the same structure as a procedure, except that:

1. A function heading must include a RETURN clause
   - The RETURN clause specifies the data type of the value that the function returns
   - A procedure heading cannot have a RETURN clause
2. In the executable part of a function, every execution path must lead to a RETURN statement
   - The PL/SQL compiler issues a compile-time warning otherwise
   - The RETURN statement is optional and not recommended in a procedure

Additional Subprogram Parts For Functions

- A function declaration can include these options:

Option	Description
DETERMINISTIC option	Helps the optimizer avoid redundant function invocations.
PARALLEL_ENABLE option	Enables the function for parallel execution, making it safe for use in slave sessions of parallel DML evaluations.
PIPELINED option	Makes a table function pipelined, for use as a row source.
RESULT_CACHE option	Stores function results in the PL/SQL function result cache.

Additional Subprogram Parts For Functions

```
CREATE OR REPLACE
FUNCTION f_pow(x IN NUMBER, y IN NUMBER)
    RETURN NUMBER
    DETERMINISTIC
    PARALLEL_ENABLE
    RESULT_CACHE
IS
BEGIN
  RETURN x**y;
END;
/

SELECT f_pow(2, 10) from dual;

DROP FUNCTION f_pow;
```


Additional Subprogram Parts For Functions

```
CREATE TYPE info_row AS OBJECT (
  id   NUMBER,
  info VARCHAR2(50)
);
/

CREATE TYPE info_tab IS TABLE OF info_row;
/

CREATE OR REPLACE FUNCTION get_info(n IN NUMBER)
  RETURN info_tab PIPELINED
AS
BEGIN
  FOR i IN 1 .. n LOOP
    PIPE ROW(info_row(i, 'Description for ' || i));
  END LOOP;
END;
/

SELECT * FROM  TABLE(get_info(10));

DROP FUNCTION get_info;
DROP TYPE info_tab;
DROP TYPE info_row;
```


Subprogram Parts Return

- RETURN Statement
  - The RETURN statement immediately ends the execution of the subprogram or anonymous block that contains it
  - A subprogram or anonymous block can contain multiple RETURN statements
- RETURN Statement in Function
  - Every execution path must lead to a RETURN statement and every RETURN statement must specify an expression
  - The RETURN statement assigns the value of the expression to the function identifier and returns control to the invoker, where execution resumes immediately after the invocation
  - In a pipelined table function, a RETURN statement need not specify an expression

Subprogram Parts Return

```
DECLARE
  x INTEGER;

  FUNCTION f (n INTEGER) RETURN INTEGER
  IS
  BEGIN
    RETURN (n*n);
  END;

BEGIN
  DBMS_OUTPUT.PUT_LINE ('f returns ' || f(2) || '. Execution returns here (1).' );
  x := f(3);
  DBMS_OUTPUT.PUT_LINE('Execution returns here (2).');
  x := &x;
  IF f(x) < 10 THEN
    DBMS_OUTPUT.PUT_LINE('Execution returns here (3).');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Execution returns here (4).');
  END IF;
END;
/
```


Subprogram Parts Return

- RETURN Statement in Procedure
  - The RETURN statement returns control to the invoker, where execution resumes immediately after the invocation
  - The RETURN statement cannot specify an expression
- RETURN Statement in Anonymous Block
  - The RETURN statement exits its own block and all enclosing blocks
  - The RETURN statement cannot specify an expression

Subprogram Parts Return

```
DECLARE
  PROCEDURE p IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Inside p');
    RETURN;
    DBMS_OUTPUT.PUT_LINE('Unreachable statement.');
  END;
BEGIN
  p;
  DBMS_OUTPUT.PUT_LINE('Control returns here.');
END;
/
```


Subprogram Parts Return

```
BEGIN
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Inside inner block.');
    RETURN;
    DBMS_OUTPUT.PUT_LINE('Unreachable statement.');
  END;
  DBMS_OUTPUT.PUT_LINE('Inside outer block.
                        Unreachable statement.');
END;
/
```


Forward Declarations

- If nested subprograms in the same PL/SQL block invoke each other, then one requires a forward declaration, because a subprogram must be declared before it can be invoked.
- A forward declaration declares a nested subprogram but does not define it
- A developer must define it later in the same block
- The forward declaration and the definition must have the same subprogram heading

Forward Declarations

```
DECLARE
  -- Declare proc1 (forward declaration)
  PROCEDURE proc1;

  -- Declarea and define proc2
  PROCEDURE proc2
  IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Call proc1');
    proc1;
  END;

  -- Define proc1
  PROCEDURE proc1
  IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Inside proc1');
  END;
BEGIN
  proc2;
END;
/
```


Subprogram Parameters

- If a subprogram has parameters, their values can differ for each invocation
- Formal parameters must be declared in the subprogram heading to have subprogram with parameters
- In each formal parameter declaration specify
  - Mandatory: the name and data type of the parameter
  - Optional: the mode and default value
- Reference the formal parameters by their names in the execution part of the subprogram

Subprogram Parameters

- When invoking the subprogram, specify the actual parameters whose values are to be assigned to the formal parameters
- **Corresponding actual and formal parameters** must have compatible **data types**
- To avoid confusion, use different names for formal and actual parameters
- Actual parameters (including default values of formal parameters) can be evaluated in any order
- LOB parameters cannot be used in a server-to-server remote procedure call (RPC)

Subprogram Parameters

```
DECLARE
  emp_num NUMBER(6) := 120;
  bonus   NUMBER(6) := 100;
  merit   NUMBER(4) := 50;

  PROCEDURE raise_salary (
    emp_id NUMBER,  -- formal parameter
    amount NUMBER   -- formal parameter
  ) IS
  BEGIN
    UPDATE employees
    SET salary = salary + amount  -- reference to formal parameter
    WHERE employee_id = emp_id;   -- reference to formal parameter
  END raise_salary;

BEGIN
  raise_salary(emp_num, bonus);          -- actual parameters
  raise_salary(emp_num, merit + bonus);  -- actual parameters
END;
/
```


Subprogram Parameters

- If the data type of a formal parameter is a constrained subtype, then:
  - If the subtype has the NOT NULL constraint, then the actual parameter inherits it
  - If the subtype has the base type VARCHAR2, then the actual parameter does not inherit the size of the subtype
  - If the subtype has a numeric base type, then the actual parameter inherits the range of the subtype, but not the precision or scale
- In a function
  - The clause RETURN datatype declares a hidden formal parameter
  - The statement RETURN value specifies the corresponding actual parameter
  - The same rules apply for subtype data types in RETURN

Subprogram Parameters

```
DECLARE
  -- Declare subtype
  SUBTYPE license IS VARCHAR2(7) NOT NULL;
  n license := 'DLLLDDD';

  PROCEDURE p (x License) IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE(x);
  END;

BEGIN
  p('1ABC123456789');  -- Succeeds; size is not inherited
  p(NULL);             -- Raises error; NOT NULL is inherited
END;
/
```


Subprogram Parameters

```
DECLARE
  FUNCTION test (p INTEGER) RETURN INTEGER IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('p = ' || p);
    RETURN p;
  END test;

BEGIN
  -- actual parameter and return value inherit
  -- only range from subtype
  DBMS_OUTPUT.PUT_LINE('test(p) = ' || test(0.66));
END;
/
```


Subprogram Parameters

```
DECLARE
  FUNCTION test (p NUMBER) RETURN NUMBER IS
    -- Implicitly converts p to INTEGER
    q INTEGER := p;
  BEGIN
    -- Display q, not p
    DBMS_OUTPUT.PUT_LINE('p = ' || q);
    -- Return q, not p
    RETURN q;
  END test;
BEGIN
  DBMS_OUTPUT.PUT_LINE('test(p) = ' || test(0.66));
END;
/
```

Subprogram Parameters Passing Method

- The PL/SQL compiler has two ways of passing an actual parameter to a subprogram:
  - By reference
    - The compiler passes the subprogram a pointer to the actual parameter
    - The actual and formal parameters refer to the same memory location
  - By value
    - The compiler assigns the value of the actual parameter to the corresponding formal parameter
    - The actual and formal parameters refer to different memory locations
- The method by which the compiler passes a specific actual parameter depends on its mode

Subprogram Parameters Passing Method

- If necessary, the compiler implicitly converts the data type of the actual parameter to the data type of the formal parameter
- Avoid implicit data conversion, in either of these ways:
  - Declare the variables that you intend to use as actual parameters with the same data types as their corresponding formal parameters
  - Explicitly convert actual parameters to the data types of their corresponding formal parameters, using the SQL conversion functions described in Oracle Database SQL Language Reference

Subprogram Parameters Passing Method

```
CREATE OR REPLACE PROCEDURE p (n NUMBER)
AUTHID DEFINER
IS
BEGIN
  DBMS_OUTPUT.PUT_LINE(n);
END;
/

DECLARE
  x NUMBER      := 1;
  y VARCHAR2(1) := '1';
BEGIN
  -- No conversion needed
  p(x);
  -- y implicitly converted from VARCHAR2 to NUMBER
  p(y);
  -- y explicitly converted from VARCHAR2 to NUMBER
  p(TO_NUMBER(y));
END;
/
```


Subprogram Parameters Mode

- The **mode** of a formal parameter determines its behavior

Parameter Mode	Is Default?	Role
IN	Default mode	Passes a value to the subprogram.
OUT	Must be specified.	Returns a value to the invoker.
IN OUT	Must be specified.	Passes an initial value to the subprogram and returns an updated value to the invoker.

Subprogram Parameters Mode Characteristics

Mode	Formal Parameter	Actual Parameter	Passed by Reference?
IN	**Formal parameter acts like a constant**: When the subprogram begins, its value is that of either its actual parameter or default value, and the subprogram cannot change this value.	Actual parameter can be a constant, initialized variable, literal, or expression.	Actual parameter is passed by reference.
OUT	**Formal parameter is initialized to the default value of its type.** The default value of the type is NULL except for a record type with a non-NULL default value. When the subprogram begins, the formal parameter has its initial value regardless of the value of its actual parameter. Oracle recommends that the subprogram assign a value to the formal parameter.	If the default value of the formal parameter type is NULL, then the actual parameter must be a variable whose data type is not defined as NOT NULL.	By default, actual parameter is passed by value. If you specify NOCOPY, it might be passed by reference.
IN OUT	**Formal parameter acts like an initialized variable**: When the subprogram begins, its value is that of its actual parameter. Oracle recommends that the subprogram update its value.	Actual parameter must be a variable (typically, it is a string buffer or numeric accumulator).	By default, actual parameter is passed by value (in both directions). If you specify NOCOPY, it might be passed by reference.

Subprogram Parameters Mode

- Do not use OUT and IN OUT for function parameters
  - Ideally, a function takes zero or more parameters and returns a single value
  - A function with IN OUT parameters returns multiple values and has side effects
- Regardless of how an OUT or IN OUT parameter is passed:
  - If the subprogram exits successfully, then the value of the actual parameter is the final value assigned to the formal parameter
  - The formal parameter is assigned at least one value (the initial value)
  - If the subprogram ends with an exception, then the value of the actual parameter is undefined
  - Formal OUT and IN OUT parameters can be returned in any order

Subprogram Parameters Mode

- When an OUT or IN OUT parameter is passed by reference:
  - The actual and formal parameters refer to the same memory location
  - If the subprogram changes the value of the formal parameter, the change shows immediately in the actual parameter

Subprogram Parameters Mode

```
CREATE OR REPLACE PROCEDURE p (
  a        PLS_INTEGER,  -- IN by default
  b     IN PLS_INTEGER,
  c    OUT PLS_INTEGER,
  d IN OUT PLS_INTEGER
)
IS
BEGIN
  DBMS_OUTPUT.PUT_LINE('Inside procedure p:');
  -- Print values of parameters:
  DBMS_OUTPUT.PUT_LINE('IN a = '     || NVL(TO_CHAR(a), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('IN b = '     || NVL(TO_CHAR(b), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('OUT c = '    || NVL(TO_CHAR(c), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('IN OUT d = ' || TO_CHAR(d));
  -- Can reference IN parameters a and b,
  -- but cannot assign values to them.
  c := a + 10;  -- Assign value to OUT parameter
  d := b * 10;  -- Assign value to IN OUT parameter
END;
/
```

Subprogram Parameters Mode

```
DECLARE
  aa  CONSTANT PLS_INTEGER := 1;
  bb  PLS_INTEGER := 2;
  cc  PLS_INTEGER := 3;
  dd  PLS_INTEGER := 4;
  ee  PLS_INTEGER;
  ff  PLS_INTEGER := 5;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Before invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('aa = ' || NVL(TO_CHAR(aa), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('bb = ' || NVL(TO_CHAR(bb), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('cc = ' || NVL(TO_CHAR(cc), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('dd = ' || TO_CHAR(dd));

  p (aa, -- constant
     bb, -- initialized variable
     cc, -- initialized variable
     dd  -- initialized variable
  );

  DBMS_OUTPUT.PUT_LINE('After invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('aa = ' || NVL(TO_CHAR(aa), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('bb = ' || NVL(TO_CHAR(bb), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('cc = ' || NVL(TO_CHAR(cc), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('dd = ' || TO_CHAR(dd));

  DBMS_OUTPUT.PUT_LINE('Before invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('ee = ' || NVL(TO_CHAR(ee), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('ff = ' || TO_CHAR(ff));

  p (1,
     (bb+3)*4, -- expression
     ee,       -- uninitialized variable
     ff        -- initialized variable
   );

  DBMS_OUTPUT.PUT_LINE('After invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('ee = ' || NVL(TO_CHAR(ee), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('ff = ' || TO_CHAR(ff));
END;
/
```


Subprogram Parameters Aliasing

- Aliasing is having two different names for the same memory location
- If a stored item is visible by more than one path and if the item is changed by one path, then the change propagates to all paths
- Subprogram parameter aliasing always occurs when
  - The compiler passes an actual parameter by reference
  - A subprogram has cursor variable parameters
- When the compiler passes an actual parameter by reference:
  - The actual and formal parameters refer to the same memory location
  - The change shows immediately in the actual parameter if the subprogram changes the value of the formal parameter

Subprogram Parameters Aliasing

- The compiler always passes IN parameters by reference:
  - The resulting aliasing cannot cause problems
  - Subprograms cannot assign values to IN parameters
- If you specify NOCOPY for an OUT or IN OUT parameter
  - The compiler might pass an OUT or IN OUT parameter by reference
  - NOCOPY is only a hint: each time the subprogram is invoked, the compiler decides, silently, whether to obey or ignore NOCOPY.
  - Aliasing can occur for one invocation but not another, making subprogram results indeterminate

Subprogram Parameters Aliasing

- If the actual parameter is a global variable, then an assignment to the formal parameter might show in the global parameter
- If the same variable is the actual parameter for two formal parameters, then an assignment to either formal parameter might show immediately in both formal parameters
- If the actual parameter is a package variable, then an assignment to either the formal parameter or the package variable might show immediately in both the formal parameter and the package variable
- If the subprogram is exited with an unhandled exception, then an assignment to the formal parameter might show in the actual parameter

Subprogram Parameters Aliasing

```
DECLARE
  n NUMBER := 10;

  PROCEDURE p (
    n1 IN NUMBER,
    n2 IN OUT NUMBER,
    n3 IN OUT NOCOPY NUMBER
  ) IS
  BEGIN
    -- actual parameter is 20 only after procedure succeeds
    n2 := 20;
    -- actual parameter value is still 10
    DBMS_OUTPUT.put_line(n1);
    -- might change actual parameter immediately
    n3 := 30;
    -- actual parameter value is either
    -- 10 (ignore NOCOPY)
    -- 30 (obey NOCOPY)
    DBMS_OUTPUT.put_line(n1);
  END;

BEGIN
  p(n, n, n);
  DBMS_OUTPUT.put_line(n);
END;
/
```


Subprogram Parameters Aliasing with Cursor Variable Parameters

- Cursor variable parameters are pointers
- If a subprogram assigns one cursor variable parameter to another, they refer to the same memory location
- This aliasing can have unintended results

Subprogram Parameters Aliasing with Cursor Variable Parameters

```
DECLARE
  c1 SYS_REFCURSOR;
  c2 SYS_REFCURSOR;

  PROCEDURE get_emp_data (
    emp_cv1 IN OUT SYS_REFCURSOR,
    emp_cv2 IN OUT SYS_REFCURSOR
  )
  IS
    emp_rec employees%ROWTYPE;
  BEGIN
    OPEN emp_cv1 FOR SELECT * FROM employees;
    -- now both variables refer to same location
    emp_cv2 := emp_cv1;
    -- fetches first row of employees
    FETCH emp_cv1 INTO emp_rec;
    -- fetches second row of employees
    FETCH emp_cv1 INTO emp_rec;
    -- fetches third row of employees
    FETCH emp_cv2 INTO emp_rec;
    -- closes both variables
    CLOSE emp_cv1;
    -- causes error when get_emp_data is invoked
    -- FETCH emp_cv2 INTO emp_rec;
  END;
BEGIN
  get_emp_data(c1, c2);
END;
/
```


Subprogram Parameters Default Values

- A formal IN parameter can have a default
- A formal parameter with a default value is called an optional parameter (its corresponding actual parameter is optional in a subprogram invocation)
- If the actual parameter is omitted, then the invocation assigns the default value to the formal parameter
- A formal parameter with no default value is called a required parameter (its corresponding actual parameter is required in a subprogram invocation)
- Omitting an actual parameter does not make the value of the corresponding formal parameter NULL
- To make the value of a formal parameter NULL, specify NULL as either the default value or the actual parameter

Subprogram Parameters
Positional notation

- When invoking a subprogram, a user can specify the actual parameters using either
- Positional notation
  - Specify the actual parameters in the same order as the formal parameters are declared
  - You can omit trailing optional parameters
  - Specifying actual parameters in the wrong order can cause problems that are hard to detect, especially if the actual parameters are literals.
  - Subprogram invocations must change if the formal parameter list changes, unless the list only acquires new trailing optional parameters
  - Reduced code clarity and maintainability
  - Not recommended if the subprogram has a large number of parameters

Subprogram Parameters
Named notation

- Specify the actual parameters in any order, using this syntax:
  formal => actual
- Where
  - **formal** is the name of the formal parameter
  - **actual** is the actual parameter
- You can omit any optional parameters
- There is no wrong order for specifying actual parameters
- Subprogram invocations must change only if the formal parameter list acquires new required parameters
- Recommended when you invoke a subprogram defined or maintained by someone else

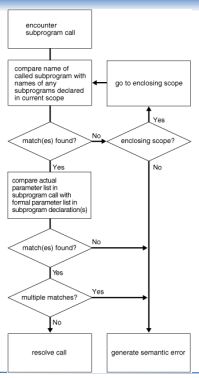
Subprogram Parameters
Mixed notation

- Start with positional notation, then use named notation for the remaining parameters
- In the positional notation: you can omit trailing optional parameters
- In the named notation: you can omit any optional parameters
- Convenient when you invoke a subprogram that has required parameters followed by optional parameters, and you must specify only a few of the optional parameters.
- In the positional notation, the wrong order can cause problems that are hard to detect, especially if the actual parameters are literals
- Changes to the formal parameter list might require changes in the positional notation

Subprogram Parameters
Default Values and Notations

```
DECLARE
  FUNCTION f_pow (
    x IN number,
    y IN number := 2,
    z IN number default 1
  ) RETURN NUMBER IS
  BEGIN
    RETURN (x ** y) ** z;
  END f_pow;
BEGIN
  DBMS_OUTPUT.PUT_LINE(f_pow(2));
  DBMS_OUTPUT.PUT_LINE(f_pow(2, 4));
  DBMS_OUTPUT.PUT_LINE(f_pow(2, 4, 4));
  DBMS_OUTPUT.PUT_LINE(f_pow(z => 2, x => 4));
  DBMS_OUTPUT.PUT_LINE(f_pow(3, 3, z => 4));
  DBMS_OUTPUT.PUT_LINE(f_pow(5, z => 4));
  DBMS_OUTPUT.PUT_LINE(f_pow(z => 3, x => 3, y => 4));
END;
/
```


Subprogram Invocation Resolution



Subprogram Invocation Resolution

- When the PL/SQL compiler encounters a subprogram invocation, it searches for a matching subprogram declaration
  - First in the current scope
  - If necessary, in successive enclosing scopes
- A declaration and invocation match if their subprogram names and parameter lists match
- The parameter lists match if each required formal parameter in the declaration has a corresponding actual parameter in the invocation
- If the compiler finds no matching declaration for an invocation, then it generates a semantic error

Overloaded Subprograms

- PL/SQL permits to overload nested subprograms, package subprograms, and type methods
- The same name can be used for several different subprograms if their formal parameters differ in name, number, order, or data type family
- A data type family is a data type and its subtypes
- If formal parameters differ only in name, then named notation must be used to specify the corresponding actual parameters

Overloaded Subprograms

- Subprograms can be overloaded if their formal parameters differ only in numeric data type
- This technique is useful in writing mathematical application programming interfaces (APIs) (i.e., several versions of a function can use the same name, and each can accept a different numeric type)
- To avoid problems or unexpected results when passing parameters to such overloaded subprograms:
  - Ensure that the expected version of a subprogram is invoked for each set of expected parameters
  - Qualify numeric literals and use conversion functions to make clear what the intended parameter types are

Overloaded Subprograms

- PL/SQL looks for matching numeric parameters in this order:
  - PLS_INTEGER (or BINARY_INTEGER, an identical data type)
  - NUMBER
  - BINARY_FLOAT
  - BINARY_DOUBLE
  - A VARCHAR2 value can match a NUMBER, BINARY_FLOAT, or BINARY_DOUBLE parameter.
- PL/SQL uses the first overloaded subprogram that matches the supplied parameters
- If PL/SQL must convert a parameter to another data type, it first tries to convert it to a higher data type

Overloaded Subprograms DO NOT DO

- Cannot overload these subprograms:
  - Standalone subprograms
  - Subprograms whose formal parameters differ only in mode; e.g.:
    PROCEDURE s (p IN VARCHAR2) IS …
    PROCEDURE s (p OUT VARCHAR2) IS …
  - Subprograms whose formal parameters differ only in subtype; e.g.:
    PROCEDURE s (p INTEGER) IS …
    PROCEDURE s (p REAL) IS …
    - INTEGER and REAL are subtypes of NUMBER, so they belong to the same data type family.
  - Functions that differ only in return value data type, even if the data types are in different families; e.g.:
    FUNCTION f (p INTEGER) RETURN BOOLEAN IS …
    FUNCTION f (p INTEGER) RETURN INTEGER IS …

Overloaded Subprograms Overload Errors

- The PL/SQL compiler catches overload errors as soon as it determines that it cannot tell which subprogram was invoked
- When subprograms have identical headings, the compiler catches the overload error at the compilation time of
  - The subprograms themselves (if they are nested)
  - The package specification that declares them
  - An ambiguous invocation of a subprogram

Overloaded Subprograms Overload

```
DECLARE
  TYPE date_tab_typ IS TABLE OF DATE   INDEX BY PLS_INTEGER;
  TYPE num_tab_typ  IS TABLE OF NUMBER INDEX BY PLS_INTEGER;

  hiredate_tab  date_tab_typ;
  sal_tab       num_tab_typ;

  PROCEDURE initialize (tab OUT date_tab_typ, n INTEGER) IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Invoked first version');
    FOR i IN 1..n LOOP
      tab(i) := SYSDATE;
    END LOOP;
  END initialize;

  PROCEDURE initialize (tab OUT num_tab_typ, n INTEGER) IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Invoked second version');
    FOR i IN 1..n LOOP
      tab(i) := 0.0;
    END LOOP;
  END initialize;

BEGIN
  initialize(hiredate_tab, 50);
  initialize(sal_tab, 100);
END;
/
```


Recursive Subprograms

- A recursive subprogram invokes itself
- Recursion is a powerful technique for simplifying an algorithm
- A recursive subprogram must have at least two execution paths
  1. One leading to the recursive invocation
  2. One leading to a terminating condition
  - Without the latter, recursion continues until PL/SQL runs out of memory and raises the predefined exception STORAGE_ERROR
- Each recursive invocation of a subprogram creates an instance of
  - Each item that the subprogram declares
  - Each SQL statement that the subprogram executes
- A recursive invocation inside a cursor FOR LOOP statement, or between an OPEN or OPEN FOR statement and a CLOSE statement
  - Opens another cursor at each invocation
  - Might cause the number of open cursors to exceed the limit set by the database initialization parameter OPEN_CURSORS

Recursive Subprograms

```
CREATE OR REPLACE FUNCTION factorial(n POSITIVE)
RETURN POSITIVE AUTHID DEFINER
IS
BEGIN
  IF n = 1 THEN
    -- terminating condition
    RETURN n;
  ELSE
    -- recursive invocation
    RETURN n * factorial(n-1);
  END IF;
END;
/

BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i || '! = ' || factorial(i));
  END LOOP;
END;
/
```


Subprograms Side Effects

- A subprogram has side effects if it changes anything except the values of its own local variables.
- A subprogram that changes any of the following has side effects:
  - Its own OUT or IN OUT parameter
  - A global variable
  - A public variable in a package
  - A database table
  - The database
  - The external state
- Side effects can
  - Prevent the parallelization of a query
  - Yield order-dependent (and therefore, indeterminate) results
  - Require that package state be maintained across user sessions
- Minimizing side effects is especially important when defining a result-cached function or a stored function for SQL statements to invoke.

PL/SQL Function Result Cache

- When a PL/SQL function has the RESULT_CACHE option, its results are cached in the shared global area (SGA) so sessions connected to the same instance can reuse these results when available
- Oracle Database
  - Automatically detects all data sources (tables and views) that are queried while a result-cached function is running
  - Changes to any of these data sources are committed, the cached result becomes invalid across all instances
- The best candidates for result-caching are functions that are invoked frequently but depend on information that changes infrequently or never
- To make a function result-cached, include the RESULT_CACHE clause in the function declaration and definition

PL/SQL Function Result Cache

```
CREATE OR REPLACE FUNCTION f
RETURN VARCHAR2
RESULT_CACHE
IS
  name VARCHAR2(50);
BEGIN
  SELECT first_name || ' ' || last_name
  INTO name
  FROM employees
  WHERE manager_id IS NULL;
  RETURN name;
END;
/
```


PL/SQL Functions Invoked by SQL

- To be invokable from SQL statements, a stored function (and any subprograms that it invokes) must obey the following purity rules, which are meant to control side effects:
  - The subprogram **cannot** modify any database tables when invoked from a SELECT statement or a parallelized INSERT, UPDATE, DELETE, or MERGE statement
  - The subprogram **cannot** query or modify any database tables modified by that statement when invoked from an INSERT, UPDATE, DELETE, or MERGE statement
  - The **subprogram cannot execute** any of the following SQL statements (unless PRAGMA AUTONOMOUS_TRANSACTION was specified) when invoked from a SELECT, INSERT, UPDATE, DELETE, or MERGE statement:
    - Transaction control statements (such as COMMIT)
    - Session control statements (such as SET ROLE)
    - System control statements (such as ALTER SYSTEM)
    - Database definition language (DDL) statements (such as CREATE), which are committed automatically

PL/SQL Functions Invoked by SQL

- If any SQL statement in the execution part of the function violates a rule, then a runtime error occurs when that statement is parsed
- The fewer side effects a function has, the better it can be optimized in a SELECT statement, especially if the function is declared with the option DETERMINISTIC or PARALLEL_ENABLE

```
SELECT f CEO_NAME FROM dual;
```


External Subprograms

- If a **C procedure** or **Java method** is **stored** in the database, you can **publish** it **as an external** subprogram
- To **publish** an external subprogram, **define** a stored PL/SQL **subprogram** with a **call specification**
- **Invoke** the published external subprogram by its PL/SQL name

External Subprograms

```
CREATE OR REPLACE JAVA SOURCE NAMED "Hello" AS
  /* Pure Java code */
  public class Hello {
    static public String Msg(String tail) {
      return "Hello " + tail;
    }
  }

CREATE OR REPLACE FUNCTION hello (str varchar2)
RETURN varchar
AS
  LANGUAGE JAVA NAME 'Hello.Msg(java.lang.String) return java.lang.String';
/

SELECT hello(first_name|| ' '||last_name) FROM employees;
```


External Subprograms

```
CREATE OR REPLACE JAVA SOURCE NAMED "Adjuster" AS
 import java.sql.*;
 import oracle.jdbc.driver.*;
 public class Adjuster {
   public static void raiseSalary (int empNo, float percent)
   throws SQLException {
     Connection conn = new OracleDriver().defaultConnection();
     String sql = "UPDATE employees SET salary = salary * ? WHERE employee_id = ?";
     try {
       PreparedStatement pstmt = conn.prepareStatement(sql);
       pstmt.setFloat(1, (1 + percent / 100));
       pstmt.setInt(2, empNo);
       pstmt.executeUpdate();
       pstmt.close();
     } catch (SQLException e)
          {System.err.println(e.getMessage());}
     }
 }
/
```


External Subprograms

```
CREATE OR REPLACE PROCEDURE raise_salary (
  empid NUMBER,
  pct   NUMBER
)
AS -- call specification
  LANGUAGE JAVA NAME 'Adjuster.raiseSalary (int, float)';
/

SELECT salary FROM employees WHERE employee_id=120;

BEGIN
  -- invoke Adjuster.raiseSalary by PL/SQL name
  raise_salary(120, 10);
END;
/

SELECT salary FROM employees WHERE employee_id=120;
```


Packages

- A package is a schema object that groups logically related PL/SQL:
  - Types
  - Variables and constants
  - Subprograms
  - Cursors
  - Exceptions
- A package is compiled and stored in the database
- Applications can share the contents of a package
- A package contains two parts
  - A package specification (mandatory)
  - A package body (not mandatory)

Packages

- A package always has a specification where public items are declared
- Public items can be referenced from outside the package
- The package must also have a body if the public items include
  - Cursors (the cursors' queries should be defined in the body)
  - Subprograms (the code for public subprograms are defined in the body)
- The body can also declare and define private items that cannot be referenced from outside the package
- The body can have
  - An initialization part containing statements to initialize variables and do other one-time setup steps
  - An exception-handling part
- The body can be changed without changing
  - The specification
  - The references to the public items

Packages

- Package subprogram can be mapped to an external Java or C subprogram in either the package specification or package body
- The external subprogram name, parameter types, and return type are mapped to their SQL counterparts using a **call specification**
- A call specification
  - Declares a Java method or a C language subprogram so that it can be invoked from PL/SQL
  - Tells the database which Java method, or which named subprogram in which shared library, to invoke when an invocation is made
  - Tells the database what type conversions to make for the arguments and return value
  - Use the SQL CALL statement to invoke such a method or subprogram

Packages

- The AUTHID clause of the package specification determines whether the subprograms and cursors in the package run with the privileges of their definer (the default) invoker
  - Their unqualified references to schema objects are resolved in the schema of the definer or invoker

CREATE OR REPLACE PACKAGE pck_name AUTHID {CURRENT_USER | DEFINER}

  - CURRENT_USER – executes with the roles and privileges of the current user (invoker)
  - DEFINER – executes with the roles and privileges of the definer
- The ACCESSIBLE BY clause of the package specification lets developers to specify a white list of PL/SQL units that can access the package

CREATE OR REPLACE PACKAGE pck_name ACCESSIBLE BY (PROCEDURE proc_name, FUNCTION func_name, etc)

Packages

- Packages support the development and maintenance of **reliable** and **reusable code** with the following features:

1. Modularity
2. Easier Application Design
3. Hidden Implementation Details
4. Added Functionality
5. Better Performance
6. Easier to Grant Roles

Packages
Modularity

- Packages enables the encapsulation in named PL/SQL modules of logically related
  - Types
  - Variables and constants
  - Subprograms
  - Cursors
  - Exceptions
- Developers can make each package
  - Easy to understand
  - Make the interfaces between packages
    - Simple
    - Clear
    - Well defined

Packages
Easier Application Design

- When designing an application, developers need initially the interface information in the package specifications
- Developers
  - Can code and compile specifications without their bodies
  - Can compile standalone subprograms that reference the packages
  - Are not required to fully define the package bodies until the applications is ready for completion

Packages
Hidden Implementation Details

- Packages let developers
  - Share interface information in the package specification
  - Hide the implementation details in the package body
- Hiding the implementation details in the body has these advantages:
  - Implementation details can be changed without affecting the application interface
  - Application users cannot develop code that depends on implementation details that might change

Packages
Added Functionality

- Package public variables and cursors
  - **Can persist for the life of a session**
  - Can be shared by all subprograms that run in the environment
  - Permits the maintenance of data across transactions without storing them in the database

Packages
Better Performance

- Oracle Database loads the whole package into memory the first time a package subprogram is invoked
- Subsequent invocations of other subprograms in same the package require no disk I/O
- Packages prevent
  - Cascading dependencies
  - Unnecessary recompiling
- If the body of a package function is changed:
  - Oracle Database does not recompile other subprograms that invoke the function
  - Reason: these subprograms depend only on the parameters and return value that are declared in the specification

Packages
Easier to Grant Roles

- Developers and administrators can grant roles on the package, instead of granting roles on each object in the package

Package Specifications

- To create a package specification, use the CREATE PACKAGE statement

```
CREATE [OR REPLACE] PACKAGE package_name
[IS|AS]
   public declarations;
end [package_name];
```


Package Specifications

- A package specification declares public items
  - The scope of a public item is the schema of the package
  - A public item is visible everywhere in the schema
  - To reference a public item that is in scope but not visible, qualify it with the package name **pck_name.public_item_name**
  - Each public item declaration has all the information needed to use the item
- The ACCESSIBLE BY clause must be included in the package specification to restrict the use of a package to specified PL/SQL units

Package Specifications
Public Declarations

- Types, variables, constants, subprograms, cursors, and exceptions used by multiple subprograms
  - A type defined in a package specification is either a PL/SQL user-defined subtype or a PL/SQL composite type
  - A PL/SQL composite type defined in a package specification is incompatible with an identically defined local or standalone type
- Associative array types of standalone subprogram parameters
  - An associative array type cannot be declared at the schema level
  - The type of that variable must be declared in a package specification to pass an associative array variable as a parameter to a standalone subprogram
  - Doing so makes the type available to both
    - The invoked subprogram (which declares a formal parameter of that type)
    - To the invoking subprogram or anonymous block (which declares a variable of that type)

Package Specifications
Public Declarations

- Variables that must remain available between subprogram invocations in the same session
- Subprograms that read and write public/private variables
  - Write "get" and "set" subprograms (i.e., setters and getters)
  - Provide these subprograms to discourage package users from reading and writing public variables directly
- Subprograms that invoke each other
- Overloaded subprograms
  - Are variations of the same subprogram
  - Have the same name but different formal parameters (and data type for the parameters)

Package Body

- To create a package body, use the CREATE PACKAGE BODY statement

```
CREATE [OR REPLACE] PACKAGE
BODY package_name
[IS|AS]
   public definitions;
   private declarations;
   private definitions;
BEGIN
EXCEPTION
END [package_name];
```


Package Body

- A package body is required if a package specification declares cursors or subprograms (otherwise, it is optional)
- The package body and package specification must be in the same schema
- Every cursor or subprogram declaration in the package specification must have a corresponding definition in the package body
- The headings of corresponding subprogram declarations and definitions must match word for word (except for white space)
- The cursors and subprograms declared in the package specification and defined in the package body are public items that can be referenced from outside the package

Package Body

- The package body can also declare and define private items
  - That cannot be referenced from outside the package
  - Are necessary for the internal workings of the package
- The body can have an initialization part with statements initialize public variables and do other one-time setup steps
  - The initialization part runs only the first time the package is referenced
  - The initialization part can include an exception handler
- The package body can be changed without changing
  - The specification
  - The references to the public items.

Package Example

```
DROP TABLE people;

CREATE TABLE people (name, salary, deptno) AS
SELECT first_name || ' ' || last_name, salary, department_id
FROM employees;


CREATE OR REPLACE PACKAGE pkg_people IS
  CURSOR c RETURN people%ROWTYPE;
  PROCEDURE bonus;
END pkg_people;
/
```

```
BEGIN
  pkg_people.bonus;
END;
/
```


Package Example

```
CREATE OR REPLACE PACKAGE BODY pkg_people IS
  CURSOR c RETURN people%ROWTYPE IS SELECT * FROM people;

  FUNCTION cbonus(sal people.salary%type,
                  idd people.deptno%type,
                  avgsal OUT float)
  RETURN number
  IS
  BEGIN
    SELECT avg(salary) INTO avgsal
    FROM people WHERE deptno=idd;
    IF avgsal > sal THEN RETURN sal*0.2;
    ELSE RETURN sal*0.1;
    END IF;
  END cbonus;

  PROCEDURE bonus
  IS
    avgsal float(10);
    bonus float(10);
  BEGIN
    FOR p IN c LOOP
      bonus := cbonus(p.salary, p.deptno, avgsal);
      DBMS_OUTPUT.PUT_LINE(p.name||' ia un bonus de '
                           || bonus || ' (' ||p.salary
                           || ' ~ ' || avgsal || ')');
    END LOOP;
    IF c%ISOPEN THEN
      CLOSE c;
    END IF;
  END bonus;
END pkg_people;
/
```


Package Example

```
CREATE OR REPLACE PACKAGE test IS
  FUNCTION power(x float) RETURN float;
  FUNCTION power(x float, y float) RETURN float;
END test;
/

CREATE OR REPLACE PACKAGE BODY test IS
  PROCEDURE power(x float, y float, z IN OUT float) IS
  BEGIN
    z := x ** y;
  END;

  FUNCTION power(x float) RETURN float IS
    rez float;
  BEGIN
    test.power(x, 2, rez);
    RETURN rez;
  END;

  FUNCTION power(x float, y float) RETURN float IS
    rez float;
  BEGIN
    test.power(x, y, rez);
    RETURN rez;
  END;
END test;
/

BEGIN
  dbms_output.put_line(test.power(2)); -- 4
  dbms_output.put_line(test.power(2, 5)); -- 32
END;
/
```


Package Instantiation and Initialization

- Oracle Database instantiates the package for that session when a session references a package item
- Every session that references a package has its own instantiation of that package
- When Oracle Database instantiates a package, it also initializes it
- Initialization includes whichever of the following are applicable:
  - Assigning initial values to public constants
  - Assigning initial values to public variables whose declarations specify them
  - Executing the initialization part of the package body

Package State

- The values of the variables, constants, and cursors that a package declares (in either its specification or body) comprise its **package state**
- If a PL/SQL package declares at least one variable, constant, or cursor, then the package is stateful (otherwise, it is stateless)
- Each session that references a package item has its own instantiation of that package.
- If the package is stateful, the instantiation includes its state

Package State

- The package state persists for the life of a session, except in these situations:
  - The package is SERIALLY_REUSABLE
  - The package body is recompiled.
  - Any of the session's instantiated packages are invalidated and revalidated
- Oracle Database treats a package as stateless if its state is constant for the life of a session (or longer).
- This is the case for a package whose items are all compile-time constants.

Packages SERIALLY_REUSABLE

- SERIALLY_REUSABLE packages let developers design applications that manage memory better for scalability
- If the package is **NOT SERIALLY_REUSABLE**
  - The package state is stored in the user global area (UGA) for each user
  - The amount of UGA memory needed increases linearly with the number of users, limiting scalability
  - The package state can persist for the life of a session, locking UGA memory until the session ends

Packages SERIALLY_REUSABLE

- If a package is SERIALLY_REUSABLE
  - The package state is stored in the work area in a small pool in the system global area (SGA)
  - The package state persists only for the life of a server call
  - After the server call, the work area returns to the pool
  - If a subsequent server call references the package, then Oracle Database reuses an instantiation from the pool
  - By reusing an instantiation
    - The package re-initializes the state
    - The changes made to the package state in previous server calls are invisible

Packages SERIALLY_REUSABLE

- Include the SERIALLY_REUSABLE pragma in the package specification and, if it exists, the package body

CREATE OR REPLACE PACKAGE pkg IS
PRAGMA SERIALLY_REUSABLE;

CREATE OR REPLACE PACKAGE BODY pkg IS
PRAGMA SERIALLY_REUSABLE;

Packages SERIALLY_REUSABLE

- The work unit is a server call for a SERIALLY_REUSABLE package
- Public variables can be used only within the work unit
- After the work unit (server call) of a SERIALLY_REUSABLE package completes, Oracle Database does the following:
  - Closes any open cursors
  - Frees some non-reusable memory (for example, memory for collection and long VARCHAR2 variables)
  - Returns the package instantiation to the pool of reusable instantiations kept for this package

Packages SERIALLY_REUSABLE

- An explicit cursor in a SERIALLY_REUSABLE package
  - Remains open until either
    - It is close explicitly
    - Its work unit (server call) ends
  - A new server call must be made to re-open the cursor
  - A server call can be different from a subprogram invocation

Package Example

```
CREATE OR REPLACE PACKAGE test IS
    FUNCTION getN RETURN NUMBER;
    PROCEDURE setN(val NUMBER := 10);
END test;
/

CREATE OR REPLACE PACKAGE BODY test IS
    n NUMBER;
    FUNCTION getN RETURN NUMBER
    IS
    BEGIN
        RETURN n;
    END getN;

    PROCEDURE setN(val NUMBER := 10)
    IS
    BEGIN
        n := val;
    END;
BEGIN
    n := 20;
END test;
/

set serveroutput on
BEGIN
    dbms_output.put_line(test.getN);
    test.setN;
    dbms_output.put_line(test.getN);
    test.setN(40);
    dbms_output.put_line(test.getN);
END;
/
```

```
CREATE OR REPLACE PACKAGE sr_test IS
    PRAGMA SERIALLY_REUSABLE;
    FUNCTION getN RETURN NUMBER;
    PROCEDURE setN(val NUMBER := 10);
END sr_test;
/

CREATE OR REPLACE PACKAGE BODY sr_test IS
    PRAGMA SERIALLY_REUSABLE;
    n NUMBER;
    FUNCTION getN RETURN NUMBER
    IS
    BEGIN
        RETURN n;
    END getN;

    PROCEDURE setN(val NUMBER := 10)
    IS
    BEGIN
        n := val;
    END;
BEGIN
    n := 20;
END sr_test;
/

set serveroutput on
BEGIN
    dbms_output.put_line(sr_test.getN);
    sr_test.setN;
    dbms_output.put_line(sr_test.getN);
    sr_test.setN(40);
    dbms_output.put_line(sr_test.getN);
END;
/
```


Package Example

```
CREATE OR REPLACE PACKAGE pkg IS
  n NUMBER;
END pkg;
/

CREATE OR REPLACE PACKAGE BODY pkg IS
BEGIN
  n := 5;
END pkg;
/

CREATE OR REPLACE PACKAGE sr_pkg IS
  PRAGMA SERIALLY_REUSABLE;
  n NUMBER;
END sr_pkg;
/

CREATE OR REPLACE PACKAGE BODY sr_pkg IS
  PRAGMA SERIALLY_REUSABLE;
BEGIN
  n := 5;
END sr_pkg;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE('pkg.n: ' || pkg.n); -- pkg.n: 5
  DBMS_OUTPUT.PUT_LINE('sr_pkg.n: ' || sr_pkg.n); -- sr_pkg.n: 5
END;
/

BEGIN
  pkg.n := 10;
  sr_pkg.n := 10;
END;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE('pkg.n: ' || pkg.n); -- pkg.n: 10
  DBMS_OUTPUT.PUT_LINE('sr_pkg.n: ' || sr_pkg.n); -- sr_pkg.n: 5
END;
/
```


Package Writing Guidelines

- Become familiar with the packages that Oracle Database supplies, and avoid writing packages that duplicate their features
- Keep your packages general so that future applications can reuse them
- Design and define the package specifications before the package bodies
- In package specifications, declare only items that must be visible to invoking programs to
  - Prevent other developers from building unsafe dependencies on your implementation details
  - Reduce the need for recompilation
  - Prevent the need to recompile any subprograms that invoke the public subprograms of the package if the package specification changes
  - Remove the need for recompiling the subprograms if only the package body changes

Package Writing Guidelines

- Declare public cursors in package specifications and define them in package bodies
  - Hides cursors' queries from package users
  - Developers can change cursors' queries without changing cursor declarations
- Assign initial values in the initialization part of the package body instead of initializing them when declared
  - The code for computing the initial values can be more complex and better documented
  - If computing an initial value raises an exception, the initialization part can handle it with its own exception handler.
  - Make helper packages available only to the API if the database applications have several PL/SQL packages

Triggers

- A trigger is a named PL/SQL unit that is stored in the database and can be invoked repeatedly
- A developer
  - Can enable and disable a trigger
  - Cannot explicitly invoke a trigger
- If the trigger is enabled, the database automatically invokes the trigger whenever its triggering event occurs
- If the trigger is disabled, the database does not invoke it
- To create a trigger, use the CREATE TRIGGER statement

Triggers

- At creation time, specify
  - The **triggering event in terms of triggering statements and the item on which they act**
  - The **timing point**, which determines whether the **trigger fires before or after** the triggering statement runs
  - Whether it **fires for the entire event or each row** that the triggering statement affects
- By default, a trigger is created in the enabled state
- The trigger is said to be "created on" or "defined on" the item, which is either a table, a view, a schema, or the database

Triggers

- A **DML trigger**
  - Is created on a table or view
  - The triggering event is composed of DML statements
- A **cross edition trigger**
  - Is a DML trigger for use for edition-based redefinition
- A **system trigger**
  - Is created on a schema or the database
  - The triggering event is composed of either DDL or database operation statements

Triggers

- A **conditional trigger**
  - Is a DML or system trigger
  - Has a WHEN clause that specifies a SQL condition that the database evaluates for each row that the triggering statement affects

Triggers

- An **INSTEAD OF trigger** is either:
  - A DML trigger created on either
    - A non-editioning view
    - A nested table column of a non-editioning view
  - A system trigger defined on a CREATE statement
  - The database fires the INSTEAD OF trigger instead of running the triggering statement

Triggers

- NOTE: When a trigger fires, tables that the trigger references might be undergoing changes made by SQL statements in other users' transactions
- SQL statements running in triggers follow the same rules that standalone SQL statements do
- Specifically:
  - Queries in the trigger see the current read-consistent materialized view of referenced tables and any data changed in the same transaction
  - Updates in the trigger wait for existing data locks to be released before proceeding

Triggers

- Triggers enable the customization of the database management system
- Triggers can be used to:
  - Automatically generate virtual column values
  - Log events
  - Gather statistics on table access
  - Modify table data when DML statements are issued against views
  - Enforce referential integrity when child and parent tables are on different nodes of a distributed database
  - Publish information about
    - Database events
    - User events
    - SQL statements to subscribing applications

Triggers

- Triggers can be used to:
  - Prevent DML operations on a table after regular business hours
  - Prevent invalid transactions
  - Enforce complex business rules that cannot be defined with constraints
  - Enforce complex referential integrity rules that cannot be defined with constraints
- Caution:
  - Triggers are not reliable security mechanisms, because they are programmatic and easy to disable
  - For high-assurance security, use Oracle Database Vault

How Triggers and Constraints Differ

- Although triggers and constraints can constrain data input, they differ significantly
- A trigger always applies to new data only
- Example: Enforce NULL constraint
  - A table column does not have a null constraint
  - A trigger can prevent a DML statement from inserting NULL values
  - But the table can contain NULL values that were inserted
    - Before the trigger was defined
    - While the trigger was disabled

How Triggers and Constraints Differ

- A constraint can apply either to new data only (like a trigger) or to both new and existing data
- Constraints behavior depend on the constraint state
- Constraints are easier to write and less error-prone than triggers that enforce the same rules
- Triggers can enforce some complex business rules that constraints cannot

Triggers

- Oracle strongly recommends to use triggers to constrain data input only in these situations:
  - To enforce referential integrity when child and parent tables are on different nodes of a distributed database
  - To enforce complex business or referential integrity rules that you cannot define with constraints

DML Triggers

- A DML trigger
  - Is created on either a table or view
  - Its triggering event is composed of the DML statements DELETE, INSERT, and UPDATE
- To create a trigger that fires in response to a MERGE statement create triggers on the INSERT and UPDATE statements to which the MERGE operation decomposes
- A DML trigger is either
  - Simple
  - Compound

DML Triggers

- A **simple DML trigger** fires at exactly one of these timing points:
  - Before the triggering statement runs
    - The trigger is called
      - A *BEFORE statement trigger*
      - *Statement-level BEFORE trigger*
  - After the triggering statement runs
    - The trigger is called
      - An *AFTER statement trigger*
      - *Statement-level AFTER trigger*

DML Triggers

- A **simple DML trigger** fires at exactly one of these timing points:
  - Before each row that the triggering statement affects
    - The trigger is called
      - A *BEFORE each row trigger*
      - *Row-level BEFORE trigger*
  - After each row that the triggering statement affects
    - The trigger is called
      - An *AFTER each row trigger*
      - *Row-level AFTER trigger*

DML Triggers

- A **compound DML trigger** created on a table or editioning view can fire at one, some, or all of the preceding timing points
- Compound DML triggers help define an approach where the actions implemented for the various timing points manage to share common data
- A simple or compound DML trigger that fires at row level can access the data in the row that it is processing
- An **INSTEAD OF DML trigger** is a DML trigger created on either a non-editioning view or a nested table column of a non-editioning view

DML Triggers

- Syntax:

```
CREATE [OR REPLACE] trigger_name
  {BEFORE | AFTER | INSTEAD OF}
  {INSERT | UPDATE [OF column_list] | DELETE
    [OR {INSERT | UPDATE [OF column_list] | DELETE} ...]}
  ON [table_name | view]
  [referencing clauses]
  [FOR EACH ROW]
  [WHEN (conditions)]
DECLARE
   trigger_variables
BEGIN
   statements
END;
/
```

DML Trigger Example

```
1 CREATE OR REPLACE TRIGGER t1
2   AFTER DELETE
3   ON employees
4   FOR EACH ROW
5 BEGIN
6   DBMS_OUTPUT.PUT_LINE('Deleting');
7 END;
8 /
```


DML Triggers
Conditional Predicates

- The triggering event of a DML trigger can be composed of multiple triggering statements
- When one of them fires the trigger, the trigger can determine which one by using conditional predicates
- A conditional predicate can appear wherever a BOOLEAN expression can appear

Conditional Predicate	TRUE if and only if:
INSERTING	An INSERT statement fired the trigger.
UPDATING	An UPDATE statement fired the trigger.
UPDATING ('column')	An UPDATE statement that affected the specified column fired the trigger.
DELETING	A DELETE statement fired the trigger.

DML Trigger
Conditional Predicates Example

```
1 CREATE OR REPLACE TRIGGER t2
2   BEFORE INSERT OR UPDATE OF salary, department_id OR DELETE
3   ON employees
4 BEGIN
5   CASE
6     WHEN INSERTING THEN
7       DBMS_OUTPUT.PUT_LINE('Inserting');
8     WHEN UPDATING('salary') THEN
9       DBMS_OUTPUT.PUT_LINE('Updating salary');
10    WHEN UPDATING('department_id') THEN
11      DBMS_OUTPUT.PUT_LINE('Updating department ID');
12    WHEN DELETING THEN
13      DBMS_OUTPUT.PUT_LINE('Deleting');
14  END CASE;
15 END;
16 /
```


Correlation Names and Pseudorecords

- A trigger that fires at row level can access the data in the row that it is processing by using correlation names
- The default correlation names are OLD, NEW, and PARENT
- To change the correlation names, use the REFERENCING clause of the CREATE TRIGGER statement

Correlation Names and Pseudorecords

- If the trigger is created on a nested table
  - OLD and NEW refer to the current row of the nested table
  - PARENT refers to the current row of the parent table
- If the trigger is created on a table or view
  - OLD and NEW refer to the current row of the table or view
  - PARENT is undefined
- OLD, NEW, and PARENT are also called pseudorecords
  - They have record structure
  - Are allowed in fewer contexts than records are

Correlation Names and Pseudorecords

- The structure of a pseudorecord is table_name%ROWTYPE, where table_name is
  - The name of the table on which the trigger is created (for OLD and NEW)
  - The name of the parent table (for PARENT)
- Syntax of a pseudorecord (in the trigger body)
  :pseudorecord_name.field_name
- In the WHEN clause of a conditional trigger
  - A correlation name is not a placeholder for a bind variable
  - Omit the colon

Correlation Names and Pseudorecords

Triggering Statement	OLD.field Value	NEW.field Value
INSERT	NULL	Post-insert value
UPDATE	Pre-update value	Post-update value
DELETE	Pre-delete value	NULL

Correlation Names and Pseudorecords

- The restrictions on pseudorecords are:
  - A pseudorecord cannot appear in a record-level operation
    - For example, the trigger cannot include this statement: **:NEW := NULL;**
  - A pseudorecord cannot be an actual subprogram parameter
    - A pseudorecord field can be an actual subprogram parameter
  - The trigger cannot change OLD field values
    - Trying to do so raises ORA-04085
  - If the triggering statement is DELETE, then the trigger cannot change NEW field values.
    - Trying to do so raises ORA-04084
  - An AFTER trigger cannot change NEW field values, because the triggering statement runs before the trigger fires.
    - Trying to do so raises ORA-04084

Correlation Names and Pseudorecords

- A BEFORE trigger can change NEW field values before a triggering INSERT or UPDATE statement puts them in the table
- If a statement triggers both a BEFORE trigger and an AFTER trigger and the BEFORE trigger changes a NEW field value, then the AFTER trigger "sees" that change

Correlation Names and Pseudorecords

```
CREATE OR REPLACE TRIGGER t3
  BEFORE DELETE OR INSERT OR UPDATE ON employees
  REFERENCING NEW AS n
  FOR EACH ROW
  WHEN (n.job_id = 'ST_CLERK' and OLD.salary < 4000 )
DECLARE
  sal_diff  NUMBER;
BEGIN
  sal_diff  := :n.salary  - :OLD.salary;
  DBMS_OUTPUT.PUT(:n.last_name || ': ');
  DBMS_OUTPUT.PUT('Old salary = ' || :OLD.salary || ', ');
  DBMS_OUTPUT.PUT('New salary = ' || :n.salary || ', ');
  DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
END;
/

update employees set salary = salary * 1.3;
```


INSTEAD OF DML Triggers

- An INSTEAD OF DML trigger is a DML trigger created on a non-editioning view, or on a nested table column of a non-editioning view.
- The database fires the INSTEAD OF trigger instead of running the triggering DML statement
- An INSTEAD OF trigger
  - Cannot be conditional (no WHEN clause)
  - Is the only way to update a view that is not inherently updatable
    - Design the INSTEAD OF trigger to determine what operation was intended and do the appropriate DML operations on the underlying tables
  - Is always a row-level trigger
  - Can read OLD and NEW values, but cannot change them

INSTEAD OF DML Triggers

```
CREATE OR REPLACE VIEW cr AS
  SELECT region_id, region_name, country_id, country_name
  FROM regions natural join countries;

CREATE OR REPLACE TRIGGER insert_cr INSTEAD OF INSERT ON cr
DECLARE
    ok number;
    duplicate_info EXCEPTION;
    PRAGMA EXCEPTION_INIT (duplicate_info, -00001);
BEGIN
  SELECT count(*) INTO ok FROM regions WHERE region_id = :new.region_id;
  IF ok=1 THEN
    IF :new.country_id IS NOT NULL THEN
      INSERT INTO countries VALUES(:new.country_id, :new.country_name, :new.region_id);
      DBMS_OUTPUT.PUT_LINE('Country added!');
    END IF;
  ELSE
    DBMS_OUTPUT.PUT_LINE('Region does not exist!');
    INSERT INTO regions VALUES(:new.region_id, :new.region_name);
    DBMS_OUTPUT.PUT_LINE('Region added!');
    IF :new.country_id IS NOT NULL THEN
      INSERT INTO countries VALUES(:new.country_id, :new.country_name, :new.region_id);
      DBMS_OUTPUT.PUT_LINE('Country added!');
    END IF;
  END IF;
EXCEPTION
    WHEN duplicate_info THEN
    RAISE_APPLICATION_ERROR(num=> -20107, msg=> 'Duplicate country');
END insert_cr;
/

INSERT INTO cr(region_id, country_id, country_name) VALUES(1, 'RO', 'Romania');

INSERT INTO cr(region_id, region_name, country_id, country_name)
VALUES(5, 'Oceania', 'AU', 'Australia');
```


INSTEAD OF DML Triggers

- An INSTEAD OF trigger with the NESTED TABLE clause
  - Fires only if the triggering statement operates on the elements of the specified nested table column of the view
  - Fires for each modified nested table element

INSTEAD OF DML Triggers

```
CREATE OR REPLACE TYPE job_line AS OBJECT(
  job_id VARCHAR2(10),
  job_title VARCHAR2(35),
  min_salary NUMBER(6),
  max_salary NUMBER(6)
);
/

CREATE OR REPLACE TYPE job_list IS TABLE OF job_line;
/

CREATE OR REPLACE VIEW dept_view AS
SELECT d.department_id, d.department_name,
  CAST (MULTISET (SELECT DISTINCT j.job_id, j.job_title, j.min_salary, j.max_salary
          FROM jobs j INNER JOIN employees e ON j.job_id = e.job_id
          WHERE e.department_id = d.department_id) AS job_list) dept_jobs
FROM departments d
WHERE EXISTS(SELECT employee_id FROM employees e WHERE e.department_id = d.department_id);

CREATE OR REPLACE TRIGGER dept_emplist_tr
  INSTEAD OF INSERT ON NESTED TABLE dept_jobs OF dept_view
  FOR EACH ROW
BEGIN
  DBMS_OUTPUT.PUT_LINE(:PARENT.department_name);
  DBMS_OUTPUT.PUT_LINE(:NEW.job_id||' '||:NEW.job_title||' '||:NEW.min_salary||' '||:NEW.max_salary);
  INSERT INTO jobs (job_id, job_title, min_salary, max_salary)
  VALUES (:NEW.job_id, :NEW.job_title, :NEW.min_salary, :NEW.max_salary);
END;
/

INSERT INTO TABLE(SELECT d.dept_jobs FROM dept_view d WHERE department_id = 10)
VALUES ('Tester', 'Super Tester', 2000, 5000);
```


Compound DML Triggers

- A compound DML trigger created on a table or editioning view can fire at multiple timing points
- Each timing point section
  - Has its own executable part
  - Has its own optional exception-handling part
  - Can access a common PL/SQL state
- The common state
  - Is established when the triggering statement starts
  - Is destroyed when the triggering statement completes, even when the triggering statement causes an error
- A compound DML trigger created on a non-editioning view is not really compound, because it has only one timing point section

Compound DML Triggers

- A compound trigger can be conditional, but not autonomous:
  - An autonomous subprogram
    - Can do SQL operations and commit or roll back those operations, without committing or rolling back the data in the main transaction
    - Uses **PRAGMA AUTONOMOUS_TRANSACTION**;
- Two common uses of compound triggers are:
  - To accumulate rows destined for a second table so that you can periodically bulk-insert them
  - To avoid the mutating-table error (ORA-04091):
    - A *mutating*-table is a table that
      - Is currently being modified by an UPDATE, DELETE, or INSERT statement
      - Might need to be updated by the effects of a declarative DELETE CASCADE referential integrity constraint

Compound DML Triggers

- A compound DML trigger created on a table or editioning view has at least one timing-point section
- If the trigger has multiple timing-point sections
  - They can be in any order
  - No timing-point section can be repeated
  - If a timing-point section is absent, then nothing happens at its timing point

Timing Point	Section
Before the triggering statement runs	BEFORE STATEMENT
After the triggering statement runs	AFTER STATEMENT
Before each row that the triggering statement affects	BEFORE EACH ROW
After each row that the triggering statement affects	AFTER EACH ROW

Compound DML Triggers

- Compound DML triggers have these restrictions:
  - OLD, NEW, and PARENT cannot appear
    - In the declarative part
    - The BEFORE STATEMENT section
    - The AFTER STATEMENT section
  - Only the BEFORE EACH ROW section can change the value of NEW
  - A timing-point section cannot handle exceptions raised in another timing-point section
  - If a timing-point section includes a GOTO statement, the target of the GOTO statement must be in the same timing-point section

Compound DML Triggers

- Syntax

```
CREATE OR REPLACE TRIGGER compound_trigger_name
FOR [INSERT|DELETE|UPDATE [OF column]
ON table COMPOUND TRIGGER
  -- Declarative Section (optional)
  -- Variables declared here have firing-statement duration.

  -- Executed before DML statement
  BEFORE STATEMENT IS
  BEGIN
    NULL;
  END BEFORE STATEMENT;

  -- Executed before each row change
  -- :NEW, :OLD are available
  BEFORE EACH ROW IS
  BEGIN
    NULL;
  END BEFORE EACH ROW;

  -- Executed aftereach row change
  -- :NEW, :OLD are available
  AFTER EACH ROW IS
  BEGIN
    NULL;
  END AFTER EACH ROW;

  -- Executed after DML statement
  AFTER STATEMENT IS
  BEGIN
    NULL;
  END AFTER STATEMENT;
END compound_trigger_name;
```


Compound DML Trigger Example

```
CREATE TABLE employee_salaries (
  employee_id NUMBER NOT NULL,
  change_date DATE   NOT NULL,
  salary NUMBER(8,2) NOT NULL,
  CONSTRAINT pk_employee_salaries PRIMARY KEY (employee_id, change_date),
  CONSTRAINT fk_employee_salaries FOREIGN KEY (employee_id)
    REFERENCES employees (employee_id) ON DELETE CASCADE
)
/
```


Compound DML Trigger Example

```
CREATE OR REPLACE TRIGGER maintain_employee_salaries
  FOR UPDATE OF salary ON employees COMPOUND TRIGGER
  -- Declarative Part:
  threshhold CONSTANT SIMPLE_INTEGER := 7;

  TYPE salaries_t IS TABLE OF employee_salaries%ROWTYPE
    INDEX BY SIMPLE_INTEGER;
  salaries  salaries_t;
  idx       SIMPLE_INTEGER := 0;

  PROCEDURE flush_array IS
    n CONSTANT SIMPLE_INTEGER := salaries.count();
  BEGIN
    FORALL j IN 1..n
      INSERT INTO employee_salaries VALUES salaries(j);
    salaries.delete();
    idx := 0;
    DBMS_OUTPUT.PUT_LINE('Flushed ' || n || ' rows');
  END flush_array;

  -- AFTER EACH ROW Section:
  AFTER EACH ROW IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('AFTER EACH ROW!');
    idx := idx + 1;
    salaries(idx).employee_id := :NEW.employee_id;
    salaries(idx).change_date := SYSTIMESTAMP;
    salaries(idx).salary := :NEW.salary;
    IF idx >= threshhold THEN
      flush_array();
    END IF;
  END AFTER EACH ROW;

  -- AFTER STATEMENT Section:
  AFTER STATEMENT IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('AFTER STATEMENT!');
    flush_array();
  END AFTER STATEMENT;
END maintain_employee_salaries;
/
```


Compound DML Trigger Example

```
UPDATE employees
  SET salary = salary * 1.05
  WHERE department_id = 50
/

SELECT employee_id, count(*) c
  FROM employee_salaries
  GROUP BY employee_id
/
```


System Triggers

- A system trigger
  - Is created on either
    - A schema
    - The database
  - Its triggering event is composed of either
    - DDL statements
    - Database operation statements
  - Fires at exactly one of these timing points:
    - Before the triggering statement runs
    - After the triggering statement runs
    - Instead of the triggering CREATE statement

Schema Triggers

- A **SCHEMA trigger**
  - Is created on a schema
  - Fires whenever the user who owns it
    - Is the current user
    - Initiates the triggering event

```
CREATE OR REPLACE TRIGGER drop_trigger
  BEFORE DROP ON hr.SCHEMA
  BEGIN
    RAISE_APPLICATION_ERROR (
      num => -20000,
      msg => 'Cannot drop object');
  END;
/
```

Database Triggers

- A **DATABASE trigger**
  - Is created on the database
  - Fires whenever any database user initiates the triggering event

```
CREATE TRIGGER log_errors
  AFTER SERVERERROR ON DATABASE
  BEGIN
    IF (IS_SERVERERROR (1017)) THEN
      NULL;  -- (substitute code that processes logon error)
    ELSE
      NULL;  -- (substitute code that logs error code)
    END IF;
  END;
/
```


INSTEAD OF CREATE Triggers

- An **INSTEAD OF CREATE trigger**
  - Is a SCHEMA trigger
  - The triggering event is a CREATE statement
  - The database fires the trigger instead of executing its triggering statement

```
CREATE OR REPLACE TRIGGER t
  INSTEAD OF CREATE ON SCHEMA
  BEGIN
    EXECUTE IMMEDIATE 'CREATE TABLE T (n NUMBER, m NUMBER)';
  END;
/
```